Group20: Kiara Ajodhaparsadh(u25395344), Nasiha Osman (u25110188), Mueez Gani (u24983439)

Task 1

A concise description of the chosen domain and problem.

CHOSEN DOMAIN: FILM PRODUCTION

TaskForge is a reusable work-processing system designed to manage the work involved in producing a film. Film production contains a large body of work that can be broken down into smaller pieces. For example, a production can be divided into phases such as Pre-Production, Production, and Post-Production, which can then be divided into scenes or departments, which in turn contain individual tasks.

The problem TaskForge solves is the difficulty of organising, processing and monitoring this hierarchical body of work while allowing production staff to: organise work into nested groups, process both individual tasks and groups uniformly, view the same work hierarchy in different ways, track the lifecycle of individual production tasks, add optional responsibilities to tasks at runtime, and modify the production structure while work is being processed. This gives us recursive part-whole hierarchy, rather than simply having a list of unrelated tasks. This allows for both individual work items and nested groups, with the hierarchy extending at least three levels below the client.

The design and ownership rationale

TaskForge uses the **Composite pattern** to represent a hierarchical film-production workflow. WorkItem provides a common abstraction for both individual tasks and groups of tasks. ProductionTask represents individual production work, while WorkGroup represents groups that can contain other WorkItem objects. ProductionPhase and Scene specialise WorkGroup to represent meaningful domain-level groupings such as PreProduction, Production, PostProduction, and individual film scenes. This recursive structure allows individual tasks and groups to be treated uniformly throughout the system.

At the top of the hierarchy, FilmProduction owns the major ProductionPhase objects through composition because these phases form an integral part of the film production. Similarly, WorkGroup owns its child WorkItem objects through composition, giving clear responsibility for the lifetime of the hierarchy and ensuring that child objects are destroyed with their owning group. ProductionTask also owns its current TaskState through composition. Each task has exactly one current state, which changes as the task moves through its lifecycle. **The State pattern** allows behaviour to depend on the current state without placing all state-specific logic inside ProductionTask.

The **Iterator pattern** separates traversal from the internal representation of the production hierarchy. WorkIterator provides the common iterator interface, while ProductionOrderIterator and PriorityIterator provide two independent views of the same hierarchy. The iterators maintain their own traversal state but do not own the work hierarchy, keeping traversal responsibilities separate from object lifetime management.

The **Decorator pattern** allows additional responsibilities to be added to work items at runtime. WorkItemDecorator wraps a WorkItem, while SafetyCheck, EquipmentCheck, and Insurance add optional behaviour. Decorators can be stacked so that multiple responsibilities can be applied to the same work item without modifying its original class. The decorators do not own the overall production hierarchy, avoiding conflicting ownership of the underlying work items.

FilmProduction is intentionally kept separate from WorkItem because it acts as the root of the system rather than representing a piece of production work itself. This keeps the recursive hierarchy focused on production phases, groups, scenes, and individual tasks.

Overall, our design is implemented to keep each pattern focused on a specific responsibility while allowing the four patterns to work together as a single film-production workflow.

Identification of the GoF participants

A. COMPOSITE

Component	WorkItem	Common abstraction for every piece of production work
Composite	WorkGroup	Represents a group that contains other WorkItems
Leaf	ProductionTask	Represents an individual piece of work
Concrete Composite	ProductionPhase(Pre-Production, production, post-production), Scene (OpeningScene, ClosingScene, ClimaxScene, postClimaxScene)	Domain-specific groups containing other work

B. ITERATOR

Iterator	**WorkIterator**	**Abstract interface for traversing work**
Concrete iterator	ProductionOrderIterator	Traverses the hierarchy in production/structural order
Concrete Iterator	PriorityIterator	Traverses tasks according to priority
Aggregate	WorkItem/ WorkGroup	Provides access to traversal without exposing its container

C. STATE PATTERN

Context	ProductionTask	Maintains the current state of a production task
State	TaskState	Common interface for task states
ConcreteState	Planned, InProgress, Blocked, Completed	

D. DECORATOR

Component	WorkItem	Common interface for the decorated object.
Concrete Component	ProductionTask (castLead, findLocation, filmScene, editScene, addEffects)	Original production task
Decorator	WorkItemDecorator	Base wrapper implementing WorkItem
ConcreteDecorator	SafetyCheck, EquipementCheck, Insurance	Adds responsibility

Rational for design decisions

Why use WorkItem? WorkItem provides a common abstraction for both individual production tasks and groups of work. This allows the Composite pattern to treat individual and grouped work uniformly.

Why use WorkGroup? Film production naturally consists of recursively nested groups. WorkGroup allows a group to contain other WorkItems, creating the required hierarchical structure.

Why use ProductionPhase and Scene? These provide concrete domain meaning to the composite hierarchy instead of creating a generic tree with no connection to the film-production problem.

Why use two iterators? Different users may need different views of the same production. ProductionOrderIterator follows the production hierarchy, while PriorityIterator identifies the most important tasks first. This gives the Iterator pattern a genuine purpose rather than simply replacing an STL loop.

Why use State? A production task naturally changes throughout its lifecycle. The State pattern lets a task behave differently when planned, in progress, blocked or completed, while also preventing invalid lifecycle transitions.

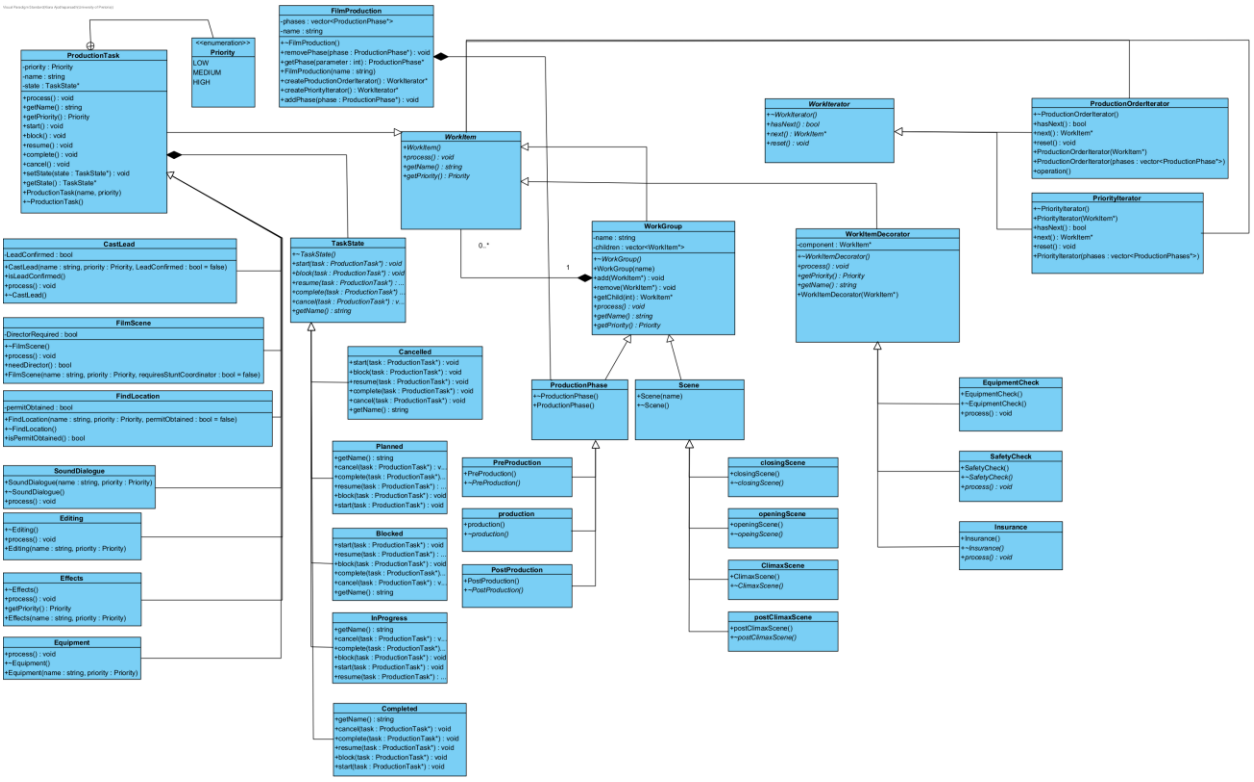
Why use Decorator? Some production tasks require additional responsibilities only in particular circumstances. Safety checks and equipment checks can therefore be added dynamically without modifying ProductionTask.

Why are decorators non-owning? The production hierarchy should have one clear owner for each work item. Making decorators non-owning prevents ownership conflicts and ensures that the hierarchy remains responsible for the lifetime of the underlying work item.

Why use composition for WorkGroup → WorkItem? The group owns the work items within it. This gives the system a clear destruction policy and makes the recursive hierarchy straightforward to manage.

How we keep the design relatively small? Since we are advised to keep the design relatively small, we deliberately do not model cameras, budgets, contracts, crew members, costumes, etc. as separate systems. They can appear as the subject of tasks, but TaskForge's responsibility is **managing the hierarchical work itself.**

THE UML CLASS DESIGN



TASK 3: Dynamic behaviour and design decisions

TaskForge uses a **snapshot traversal policy**. When an iterator is created, it captures the view of the production hierarchy at that point. If the hierarchy is subsequently modified, an existing iterator continues using its original snapshot, while a newly created iterator reflects the updated hierarchy. This makes traversal behaviour predictable and prevents an active iterator from becoming inconsistent when work is added, removed, or moved at runtime.

The system supports two meaningful and independent traversals. ProductionOrderIterator follows the production structure, while PriorityIterator provides a priority-based view of individual production tasks. Each iterator maintains its own traversal state, allowing multiple traversals of the same production to occur independently.

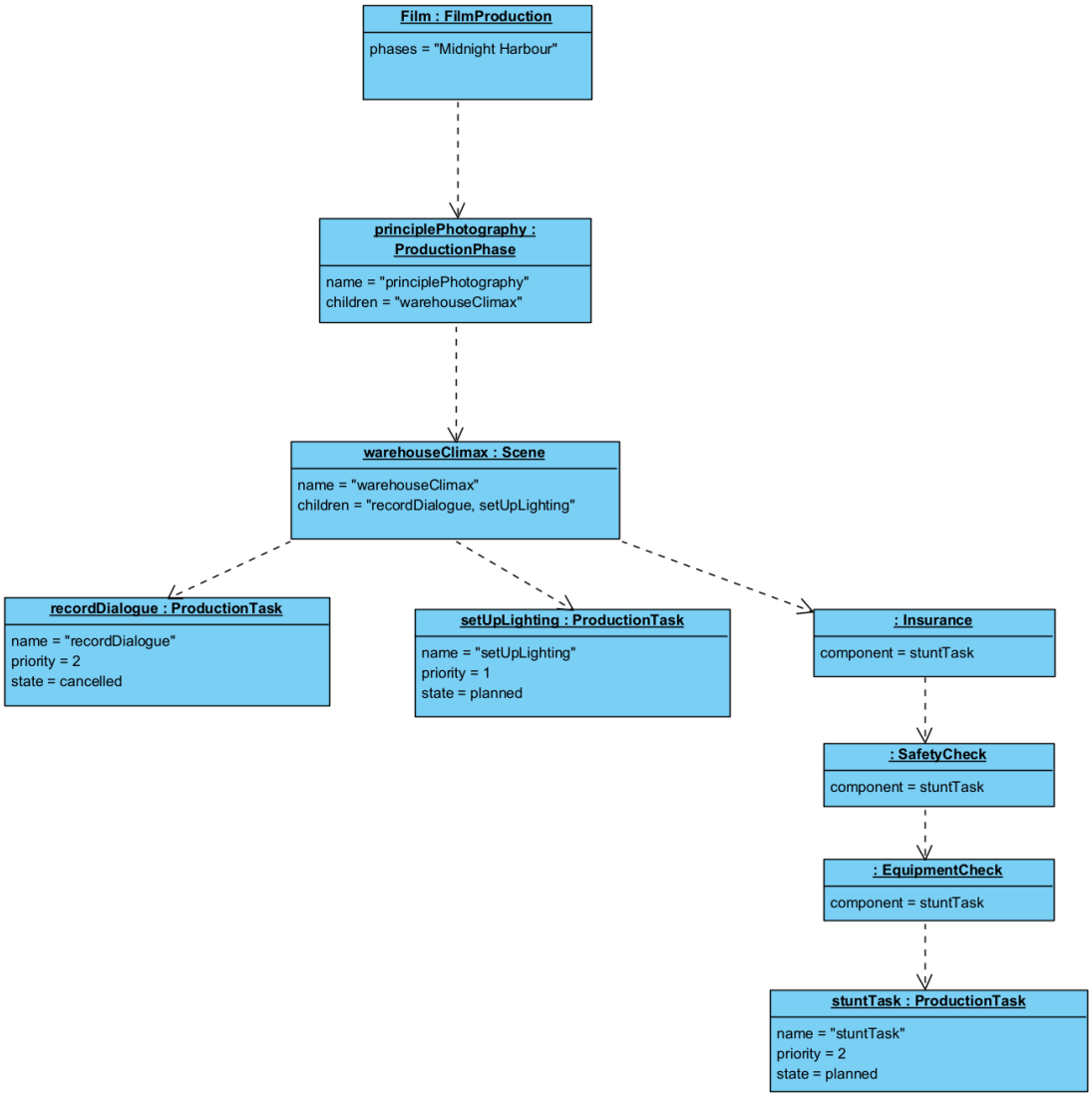
The hierarchy is intentionally recursive: a WorkGroup can contain both individual ProductionTask objects and other groups. This avoids hard-coding relationships between particular levels of the production hierarchy and allows the same Composite structure to represent phases, scenes, and tasks.

Production tasks use the State pattern to model their lifecycle. Tasks can move between Planned, InProgress, Blocked, Cancelled, and Completed according to the rules defined by their current state. Invalid transitions are rejected rather than silently changing the task to an inappropriate state.

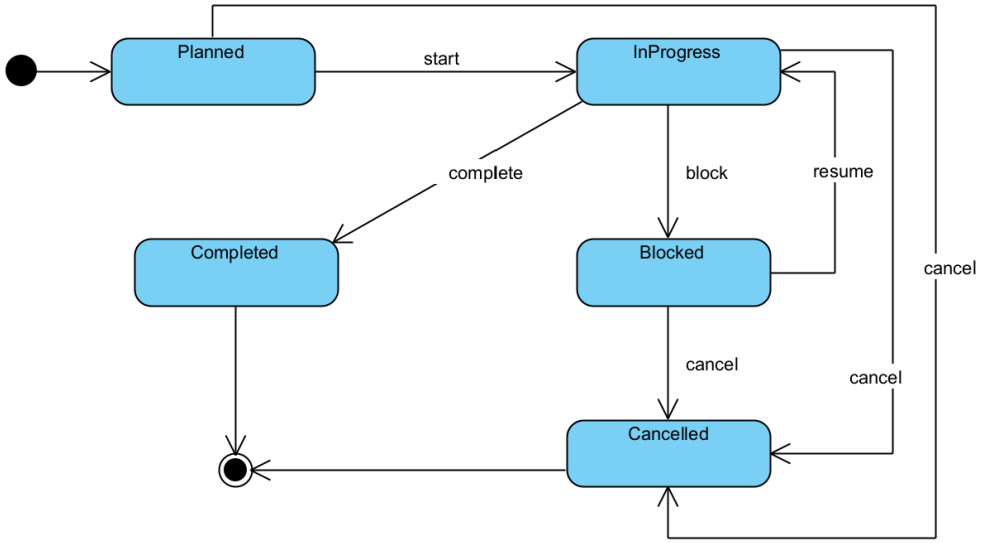
Finally, Decorators provide runtime flexibility by allowing responsibilities such as safety, equipment, and insurance checks to be added and combined without changing the underlying production task. These design decisions ensure that the UML, ownership model, runtime behaviour, and four required GoF patterns form one coherent system.

TASK 4 : UML Diagram Portfolio

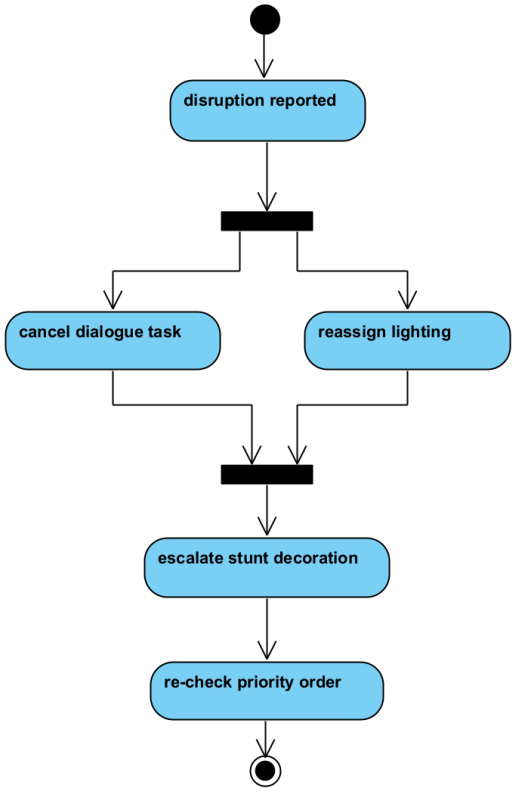
Object diagram representing FimProduction hierarchy and decorator wrapping after the execution of Scenario 2 in main.cpp



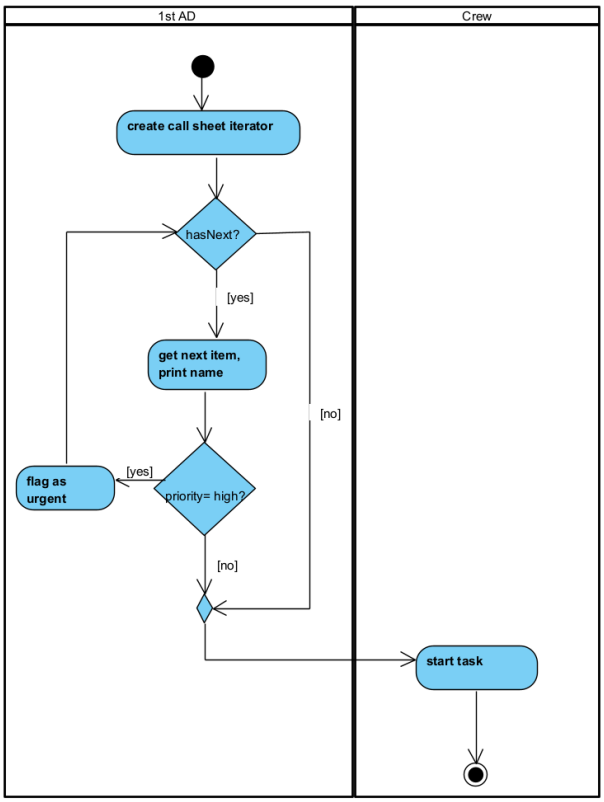
State diagram representing states of a productionTask



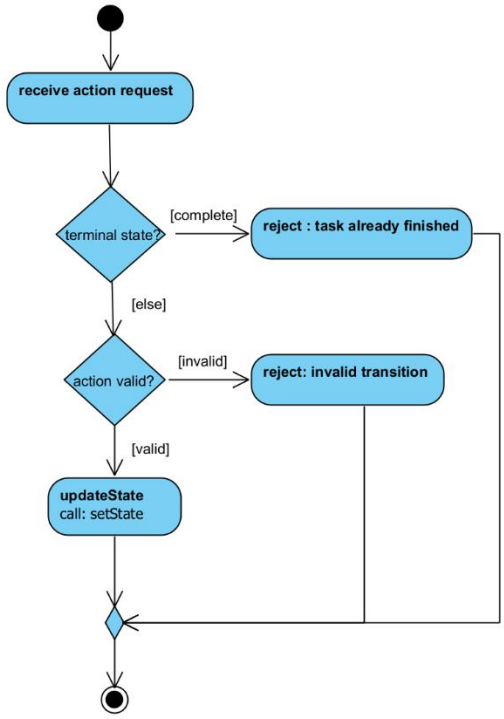
Activity diagram 1 representing the flow of activities following a disruption



Activity diagram 2 representing the flow of the Iterator class with productionTask



Activity diagram representing the flow of states in productionTask



Task 5

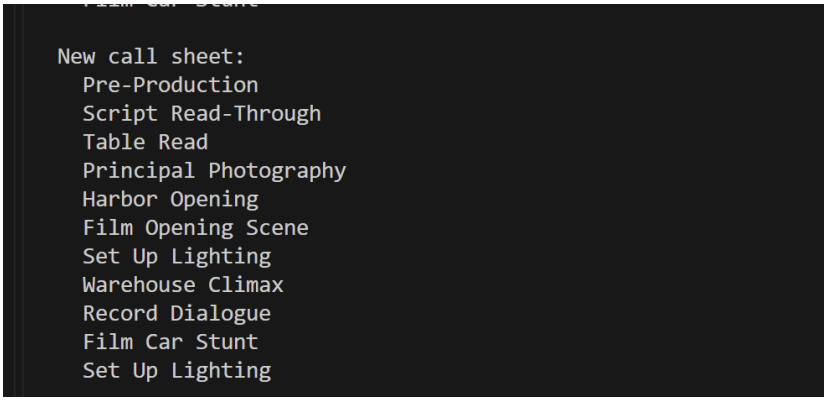
Using gdb while implementing:

```
1st AD sees:    Pre-Production

Filming the opening scene:
Film Opening Scene started.

Breakpoint 1, ProductionTask::setState (this=0x55555557d950,
    newState=0x55555557dbb0) at src/ProductionTask.cpp:65
65          TaskState* oldState = state;
(gdb) print name
$1 = "Film Opening Scene"
(gdb) print state->getName()
$2 = "Planned"
(gdb) print newState->getName()
$3 = "In Progress"
(gdb) continue
Continuing.
Film Opening Scene has been blocked.

Breakpoint 1, ProductionTask::setState (this=0x55555557d950,
    newState=0x55555557d9d0) at src/ProductionTask.cpp:65
65          TaskState* oldState = state;
(gdb) print state->getName()
$4 = "In Progress"
(gdb) print newState->getName()
$5 = "Blocked"
(gdb) continue
Continuing.
Cannot complete Film Opening Scene while it is blocked.
Film Opening Scene has resumed.
```

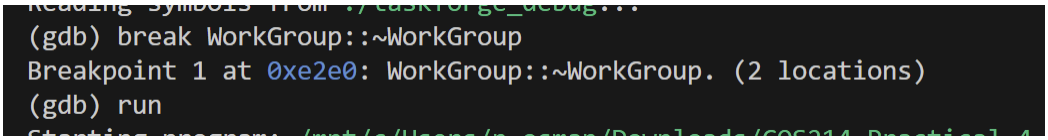
```
Breakpoint 1, ProductionTask::setState (this=0x55555557d950,
    newState=0x55555557dbb0) at src/ProductionTask.cpp:65
65          TaskState* oldState = state;
(gdb) print state->getName()
$6 = "Blocked"
(gdb) print newState->getName()
$7 = "In Progress"
(gdb) continue
Continuing.
Film Opening Scene has been completed.

Breakpoint 1, ProductionTask::setState (this=0x55555557d950,
    newState=0x55555557d9d0) at src/ProductionTask.cpp:65
65          TaskState* oldState = state;
(gdb) print state->getName()
$8 = "In Progress"
(gdb) print newState->getName()
$9 = "Completed"
(gdb)
```

An issue that was encountered:

taskforge builds cleanly but crashes with a segmentation fault at program shutdown, after printing all of Scenario 2's output. A subtler tell shows up earlier too: the "new call sheet" and the final priority listing both print "Set Up Lighting" twice.



In the weather-disruption scenario, the task added Set Up Lighting to warehouseClimax but never called harborOpening->remove(...) first. The same ProductionTask* was now sitting in two WorkGroup::children vectors. Both are WorkGroup, both own their children, and both delete every child in their destructor. The pointer gets deleted twice.



```
Breakpoint 1.1, WorkGroup::~WorkGroup (this=0x55555557d850,
    __in_chrg=<optimized out>) at src/WorkGroup.cpp:11
11      WorkGroup::~WorkGroup()
(gdb) print name
$7 = "Harbor Opening"
(gdb) print children
$8 = std::vector of length 2, capacity 2 = {0x55555557d950, 0x55555557d9f0}
(gdb) continue
Continuing.

Breakpoint 1.1, WorkGroup::~WorkGroup (this=0x55555557d8a0,
    __in_chrg=<optimized out>) at src/WorkGroup.cpp:11
11      WorkGroup::~WorkGroup()
(gdb) print name
$9 = "Warehouse Climax"
(gdb) print children
$10 = std::vector of length 3, capacity 4 = {0x55555557da30, 0x55555557d9f0,
  0x55555557da70}
(gdb) continue
Continuing.

Program received signal SIGSEGV, Segmentation fault.
0x000055555556234e in WorkGroup::~WorkGroup (this=0x55555557d8a0,
    __in_chrg=<optimized out>) at src/WorkGroup.cpp:15
15              delete item;
```

The fix:

```
world.harborOpening->remove(world.setUpLighting);

world.warehouseClimax->add(world.setUpLighting);
```

Removing from the old owner before adding to the new one restores single ownership.
After the fix: exit code 0, Set Up Lighting appears exactly once everywhere.

After the fix and final program run:

```
==2063==
==2063== HEAP SUMMARY:
==2063==     in use at exit: 0 bytes in 0 blocks
==2063==   total heap usage: 107 allocs, 107 frees, 78,031 bytes allocated
==2063==
==2063== All heap blocks were freed -- no leaks are possible
==2063==
==2063== For lists of detected and suppressed errors, rerun with: -s
==2063== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

TASK 6: Docker and GitHub Workflow

The team used GitHub branches and pull requests to divide and integrate the work. Mueez implemented the Task 2 design patterns and runtime testing on the mueez branch, with commit 6076bd3. Kiara led Task 1 (commit e91b9b8), producing the system design and class diagram on a dedicated design branch and later merged into main, Nasiha Osman covered Tasks 3, 4 and 5 — extending runtime scenarios and implementation coverage with commit 368a55e, while additional implementation, Task 5 work, the Makefile and main program were integrated through pull requests such as PR #1 and PR #3. Changes were regularly merged into main, allowing the team to work independently while keeping a shared final version.

Docker, README, Makefile and repository configuration were also updated on main so that the project could be built, run, debugged with GDB and checked with Valgrind in a reproducible environment.

Team contribution statement

The TaskForge project was developed collaboratively by all three team members. Kiara focused primarily on task 1 contributing to designing the Film Production system and integrating the required design patterns into the system, designing the UML Class Diagram and making all design decisions. Nasiha focused primarily on task 3, 4 and 5 contributing to creating two non-trivial runtime scenarios, completing the activity and state diagrams and performing the debugging and memory investigation. Mueez focused primarily on task 2 and 6 contributing to the overall system implementation and creating the makefile, dockerfile and readme. All 3 team members reviewed and integrated each other's work, we contributed to the final system, and understand our system. The team's GitHub history reflects the development and integration of these contributions.

GITHUB REPOSITORY LINK:

https://github.com/mueezgani/COS214-Practical-4-FilmProduction.git